

VANTEC CORE

Manual de Arquitectura de Infraestructura, Despliegue Multi-Tenant y Seguridad

Documento:	Directiva Técnica de Despliegue Automatizado y Hardening de Repositorios	Versión:	1.0 (Producción Oficial)
Autor:	Dirección de Tecnología (CTO)	Fecha:	Mayo de 2026
Destinatario:	Equipo de Ingeniería y Desarrollo de Software VANTEC	Estatus:	Aprobado para Implementación

1. Resumen Ejecutivo de Infraestructura

Este documento establece los lineamientos oficiales de infraestructura y despliegue continuo para la plataforma corporativa **VANTEC CORE**. El sistema ha sido migrado exitosamente de un entorno de pruebas local a una arquitectura Cloud de alta disponibilidad y aislamiento administrada mediante un VPS dedicado e instrumentada con **Coolify** y contenedores **Docker**.

Para garantizar la portabilidad absoluta del software y permitir modelos de negocio escalables (como servicios administrados en servidores propios o réplicas en infraestructuras privadas de clientes corporativos), se prohíbe terminantemente la manipulación o inyección manual de datos a nivel de base de datos en producción. Toda la configuración inicial de arranque debe estar completamente automatizada en el ciclo de vida del código fuente.

2. Arquitectura de Datos y Modelo Multi-Tenant Real

De acuerdo con la auditoría de los modelos del sistema (`models.py`), la base de datos PostgreSQL en producción opera bajo una estructura estructurada de aislamiento de inquilinos (Multi-tenant). Las tablas clave analizadas y corregidas poseen las siguientes características técnicas en su esquema real:

Tabla: `tenants`

Almacena la separación lógica de las empresas o clientes corporativos integrados en el sistema.

- `tenant_id`: Identificador único universal (UUID asíncrono `primary_key`).

- `rfc`: Registro Federal de Contribuyentes (String de 13, único, no nulo).
- `api_key`: Llave secreta para la comunicación segura con el Agente Local (String de 100, único).
- `business_name`: Razón social de la organización (String de 200, no nulo).
- `is_active`: Flag booleano de control de acceso operativo (default: true).

Tabla: `users`

Almacena el control de credenciales y perfiles de acceso. Su mapeo exacto en código de desarrollo debe alinearse con la base de datos corporativa:

- `user_id`: UUID `as_uuid primary_key` autogenerado.
- `tenant_id`: ForeignKey apuntando a `tenants.tenant_id` con propiedad `nullable=True`.
- `username`: Nombre de usuario de acceso (String único, no nulo).
- `password_hash`: Contraseña encriptada bajo algoritmo criptográfico seguro (String de 200, no nulo).
- `email`: Correo electrónico corporativo (String único, no nulo).
- `is_active`: Estado de la cuenta (Boolean).
- `is_superuser`: Interruptor maestro de privilegios de infraestructura (Boolean).
- `rol`: Rol operativo dentro del contexto (String, ej. 'ADMIN', 'OPERADOR').



Regla Criterio de Súper Administración Global

Por diseño de software avanzado multi-tenant, el **Súper Administrador Global** (Dueño de la Plataforma) **JAMÁS** debe estar ligado a un RFC o Tenant en específico. Su columna `tenant_id` debe registrarse estrictamente como NULL, y su bandera `is_superuser` debe declararse como `True`. Esto le permite al motor de autenticación otorgarle un pase VIP global para auditar, gestionar y controlar todas las empresas alojadas en el ecosistema sin restricciones de aislamiento. El rol ADMIN con un `tenant_id` asignado queda reservado exclusivamente para los usuarios administradores locales de cada cliente.

3. Directiva de Automatización: Usuario Semilla (Database Seeding)

El equipo de desarrollo debe implementar de forma obligatoria e inmediata un mecanismo automatizado de **Database Seeding** (Siembra de datos iniciales) dentro del ciclo de arranque de la aplicación FastAPI. Esto evita la necesidad de usar DBeaver o interfaces externas para inicializar servidores nuevos, asegurando que cualquier clonación del software en infraestructuras de clientes sea plug-and-play.

3.1 Script de Inicialización Replicable (Estructura SQL Oficial)

El siguiente bloque SQL define el escenario real semilla corporativo y debe ser integrado en el código de inicialización del backend:

```
-- 🧙 SCRIPT MAESTRO DE INICIALIZACIÓN DE SÚPER ADMINISTRADOR GLOBAL (HOST MASTER)
-- Este registro habilita el usuario semilla sin ligarlo a ninguna empresa previa
INSERT INTO public.users (
    user_id,
    tenant_id,
    username,
    password_hash,
    email,
    is_active,
    is_superuser,
    rol
) VALUES (
    '9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d', -- UUID estático para control de semilla
    NULL, -- NULL absoluto: Dueño global de la
plataforma
    'admin', -- Credencial de usuario de login
    '$2b$12$EcX7V68MabH6U7KzLOnw9unW.a6AhyMv6GgMsdvJnZpW0tNfVIGe2', -- Hash
criptográfico de la contraseña '@dm1n***'
    'admin@vcore.com', -- Correo de simulación corporativa
    true, -- Cuenta activa de inmediato
    true, -- is_superuser = True (Modo Dios de
Backend)
    'ADMIN' -- Nivel administrativo global
);
```

3.2 Instrucción Técnica para Implementación en FastAPI

El equipo de backend debe programar esta inserción utilizando SQLAlchemy dentro del manejador de eventos lifespan o el decorador de arranque `@app.on_event("startup")`. El algoritmo debe seguir la lógica de validación preventiva descrita a continuación:

```
from sqlalchemy import select
from src.database.models import User
import uuid

async def seed_core_database(db_session):
    # 1. Validar si ya existe un Superadmin en la plataforma para evitar duplicidad
    query = select(User).where(User.is_superuser == True)
    result = await db_session.execute(query)
    existing_admin = result.scalar_one_or_none()

    if not existing_admin:
        # 2. Inyectar el usuario semilla global de forma automática si la plataforma
        # está virgen
        seed_user = User(
            user_id=uuid.UUID('9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d'),
            tenant_id=None,
            username="admin",
```

```
password_hash="$2b$12$EcX7V68MabH6U7KzL0nw9unW.a6AhyMv6GgMsdvJnZpw0tNfVIGe2", #
Contraseña: @dm1n***
        email="admin@vcore.com",
        is_active=True,
        is_superuser=True,
        rol="ADMIN"
    )
    db_session.add(seed_user)
    await db_session.commit()
    print("[SEEDING] Base de datos inaugurada exitosamente con el Usuario Semilla
Global.")
else:
    print("[SEEDING] La base de datos ya cuenta con usuarios administradores
registrados. Saltando siembra.")
```



Alerta Técnica Importante: Incompatibilidad de Bcrypt y Passlib

Durante las pruebas de generación de hashes locales se detectó un bug crítico en entornos modernos de desarrollo (Python 3.10+ / Bcrypt 4.0+), donde la librería heredada `passlib.context` arroja el error: `ValueError: password cannot be longer than 72 bytes`. Esto ocurre debido a una validación interna errónea de `passlib` con las versiones actuales de compilación de `bcrypt`.

Solución Obligatoria: Para la generación de hashes locales y rutinas de pruebas, se ordena al equipo de desarrollo evadir `passlib` y utilizar la librería nativa `bcrypt` directamente en terminal mediante la siguiente instrucción:

```
python -c "import bcrypt; print(bcrypt.hashpw(b'@dm1n***',
bcrypt.gensalt(12)).decode('utf-8'))"
```

4. Protocolo de Hardening y Privacidad: Clausura de GitHub

Para salvaguardar la propiedad intelectual de la empresa y evitar fugas de información sensible por la presencia de archivos históricos de configuración (`.env`) y comprobantes fiscales de prueba indexados públicamente, se ejecuta la privatización total del repositorio de código fuente.

Con el fin de no interrumpir el pipeline de despliegue continuo (CI/CD) controlado por Coolify, se prohíbe el uso de autenticación HTTPS por usuario/contraseña, y se establece el uso exclusivo de **Deploy Keys (Llaves SSH Criptográficas)**. El equipo técnico debe seguir y mantener esta configuración siguiendo este protocolo:

Fase A: Registro de la Llave en GitHub

1. Extraer la clave pública SSH generada en la instancia de Coolify en producción (Sección Keys & Tokens, llave identificada como default).

2. Acceder al repositorio oficial en GitHub: https://github.com/vantecdesarrollo-afk/Gestor_Vantec_VPS.
3. Ingresar a **Settings** (Configuración) → **Deploy keys** (Llaves de despliegue).
4. Hacer clic en **Add deploy key**, asignar el título **Coolify Production VPS**, pegar el bloque completo de la clave pública criptográfica (que inicia con `ssh-rsa` o `ecdsa-sha2...`) y confirmar el guardado.

Fase B: Privatización del Repositorio

1. En el panel de control de GitHub, ir a la pestaña **Settings** → **General**.
2. Desplazarse verticalmente hasta el final de la interfaz a la sección de criticidad **Danger Zone**.
3. Ubicarse en la opción *Change repository visibility*, hacer clic en **Change visibility** y seleccionar el estado **Make private**.
4. Confirmar escribiendo el nombre completo del repositorio para validar la restricción absoluta en internet.

Fase C: Ajuste de Enrutamiento SSH en Coolify

Una vez que el repositorio es privado, la URL pública clásica por HTTP fallará de inmediato. Se debe actualizar el cableado de red de Coolify para que use el túnel cifrado SSH:

1. Entrar a la aplicación correspondiente en el panel de Coolify.
2. Ir a la pestaña de configuración **Configuration** → **Git Source**.
3. Cambiar la ruta HTTP por el formato internacional de conexión SSH estructurado de la siguiente forma:

```
git@github.com:vantecdesarrollo-afk/Gestor_Vantec_VPS.git
```

4. En el selector inferior de llaves de seguridad, vincular explícitamente la llave SSH default dada de alta en GitHub en la Fase A y presionar **Save**.

5. Control de Cambios de Infraestructura Local (Limpieza de Caché de Git)

Para asegurar que futuras actualizaciones desde estaciones de trabajo de los desarrolladores no vuelvan a subir carpetas de datos temporales, archivos locales `.env` o directorios de CFDI (`Operacion_CFDI/`), se ordena la ejecución del siguiente script de purga en la consola local del proyecto antes de realizar cualquier commit subsecuente:

```
# 1. Asegurar la inclusión de las exclusiones en el archivo maestro de ignorados
echo "Operacion_CFDI/" >> .gitignore
echo ".env" >> .gitignore
```

```
# 2. Desvincular los directorios de la memoria caché de Git (manteniendo los archivos
intactos en local)
git rm -r --cached Operacion_CFDI/
git rm --cached .env

# 3. Empujar los cambios bajo la nueva estructura privada y segura
git commit -m "🔒 security: Hardening de privacidad de repositorio corporativo y purga
de caché"
git push origin main
```

FIN DEL MANUAL TÉCNICO - PROPIEDAD DE VANTEC CONSULTORES ES EstrictAMENTE PROHIBIDA
SU DISTRIBUCIÓN EXTRACORPORATIVA